ЛАБОРАТОРНА РОБОТА № 6

Розробка клієнтських додатків з використанням Blazor WebAssembly.

Мета: отримати навички створення та інтеграції клієнтського додатку з RESTful Web API за допомогою Blazor WebAssembly.

Хід роботи

Завдання 1. Створіть у своєму рішенні проект Blazor WebAssembly, який взаємодіятиме з WebAPI. Розробіть сторінки та необхідні компоненти для виконання CRUD-операцій з існуючими сутностями. Додайте валідацію для створених форм.

На першому етапі у межах рішення було створено новий проєкт Blazor WebAssembly (клієнтський SPA-додаток), який взаємодіє з існуючим WebAPI TicketBooking.Api за допомогою HTTP-запитів. На відміну від Blazor Server, у Blazor WebAssembly весь код виконуються безпосередньо в браузері, тому доступ до даних здійснюється виключно через REST API. Базовим URL для API було використано https://localhost:5001.

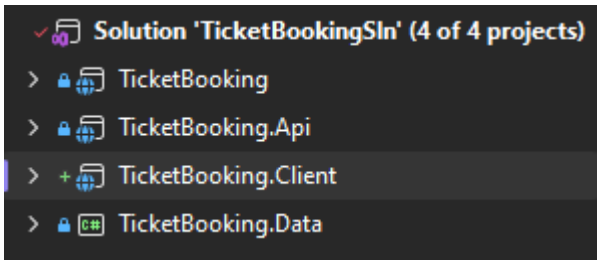

Рис. 1.1. Структура рішення TicketBooking після додавання Blazor WebAssembly клієнта.

Щоб клієнт міг звертатися до WebAPI з іншого origin, у проєкті WebAPI було увімкнено CORS. Це дозволяє браузеру виконувати запити з Blazor-клієнта до ендпоїнтів API. Налаштування виконано в Program.cs WebAPI шляхом підключення політики CORS і виклику UseCors у пайплайні.

Лістинг фрагменту коду TicketBooking.Api/Program.cs:

```
builder.Services.AddCors(options => {
      options.AddPolicy("BlazorClient", p =>
            p.AllowAnyOrigin()
             .AllowAnyMethod()
             .AllowAnyHeader());
});
…
app.UseCors("BlazorClient");
```

У проєкті TicketBooking.Client було налаштовано HttpClient з BaseAddress, що вказує на WebAPI. Завдяки цьому в Razor-компонентах можна виконувати запити відносними шляхами на кшталт api/events та api/bookings.

Лістинг коду TicketBooking.Client/Program.cs:

```
using Blazored.LocalStorage;
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using TicketBooking.Client;
using TicketBooking.Client.Infrastructure;
using TicketBooking.Client.Services;


var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

builder.Services.AddBlazoredLocalStorage();
builder.Services.AddScoped<AuthHeaderHandler>();
builder.Services.AddScoped<AuthService>();

builder.Services.AddHttpClient("Api", client => {
      client.BaseAddress = new Uri("http://localhost:5001/");
})
.AddHttpMessageHandler<AuthHeaderHandler>();

builder.Services.AddScoped(sp =>
      sp.GetRequiredService<IHttpClientFactory>().CreateClient("Api"));

await builder.Build().RunAsync();
```

Оскільки WebAPI повертає дані у вигляді DTO, у клієнтському проєкті створено відповідні класи моделей (DTO) для коректної десеріалізації JSON та для передачі даних у запитах створення/оновлення. Для Event і Booking було реалізовано окремі DTO для читання та для Create/Update. У Create/Update DTO також додано атрибути DataAnnotations, що дозволило застосовувати клієнтську валідацію в компонентах Blazor.

Лістинг коду TicketBooking.Client/Models/EventDto.cs:

```
namespace TicketBooking.Client.Models {
	public class EventDto {
		public long EventId { get; set; }
		public string Title { get; set; } = "";
		public string Description { get; set; } = "";
		public string Location { get; set; } = "";
		public DateTime StartDate { get; set; }
		public decimal BaseTicketPrice { get; set; }
		public int BookingsCount { get; set; }
	}
}
```

Лістинг коду TicketBooking.Client/Models/EventCreateUpdateDto.cs:

```
using System.ComponentModel.DataAnnotations;

namespace TicketBooking.Client.Models {
	public class EventCreateUpdateDto {
		[Required, StringLength(120)]
		public string Title { get; set; } = "";

		[Required, StringLength(1000)]
		public string Description { get; set; } = "";

		[Required, StringLength(80)]
		public string Location { get; set; } = "";

		[Required]
		public DateTime StartDate { get; set; } = DateTime.Now.AddDays(1);

		[Range(0.01, 10000000)]
		public decimal BaseTicketPrice { get; set; } = 1;
	}
}
```

Лістинг коду TicketBooking.Client/Models/BookingDto.cs:

```
namespace TicketBooking.Client.Models {
	public class BookingDto {
		public long BookingId { get; set; }
		public string CustomerName { get; set; } = "";
		public string CustomerEmail { get; set; } = "";
		public int Quantity { get; set; }
		public long EventId { get; set; }
		public DateTime CreatedAtUtc { get; set; }
```

```
            public string EventTitle { get; set; } = "";
            public DateTime EventStartDate { get; set; }
            public string EventLocation { get; set; } = "";
        }
}
```

Лістинг коду TicketBooking.Client/Models/BookingCreateUpdateDto.cs:

```
using System.ComponentModel.DataAnnotations;

namespace TicketBooking.Client.Models {
        public class BookingCreateUpdateDto {
            [Required, StringLength(100)]
            public string CustomerName { get; set; } = "";

            [Required, EmailAddress, StringLength(120)]
            public string CustomerEmail { get; set; } = "";

            [Range(1, 10)]
            public int Quantity { get; set; } = 1;

            [Range(1, long.MaxValue)]
            public long EventId { get; set; }
        }
}
```

Далі були створені сторінки-клієнти для повного CRUD над сутностями Event і Booking. Для подій реалізовано сторінку /events, яка відображає список подій, підтримує створення та редагування через форму EditForm, а також видалення з підтвердженням. Для бронювань реалізовано сторінку /bookings, де додатково використано випадаючий список подій (отриманий з api/events) для вибору EventId у формі створення/оновлення бронювання.

Лістинг коду TicketBooking.Client/Pages/Events.razor:

```
@page "/events"
@using System.Net
@using System.Net.Http.Json
@using TicketBooking.Client.Models
@inject HttpClient Http
@inject NavigationManager Nav
@inject IJSRuntime JS

<AuthGuard>
    <h3>Events</h3>

    @if (!string.IsNullOrWhiteSpace(success)) {
        <div class="alert alert-success">@success</div>
    }
    @if (!string.IsNullOrWhiteSpace(error)) {
        <div class="alert alert-danger">@error</div>
    }
```

```
    @if (events is null) {
        <p><em>Loading...</em></p>
    } else {
        <div class="row">
            <div class="col-12 col-lg-7">
                <table class="table table-striped align-middle">
                    <thead>
                        <tr>
                            <th>Title</th>
                            <th>Location</th>
                            <th>Date</th>
                            <th class="text-end">Price</th>
                            <th class="text-center">Bookings</th>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var e in events) {
                            <tr class="@(editingId == e.EventId ? "table-warning" : "")">
                                <td>@e.Title</td>
                                <td>@e.Location</td>
                                <td>@e.StartDate.ToString("yyyy-MM-dd HH:mm")</td>
                                <td class="text-end">@e.BaseTicketPrice</td>
                                <td class="text-center">@e.BookingsCount</td>
                                <td class="text-end">
                                    <button class="btn btn-sm btn-outline-secondary me-2"
                                            @onclick='() =>
Nav.NavigateTo($"/events/{e.EventId}")'>
                                        Details
                                    </button>
                                    <button class="btn btn-sm btn-outline-primary me-2"
                                            @onclick="() => StartEdit(e.EventId)">
                                        Edit
                                    </button>
                                    <button class="btn btn-sm btn-outline-danger"
                                            @onclick="() => DeleteEvent(e.EventId)">
                                        Delete
                                    </button>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>

            <div class="col-12 col-lg-5">
                <div class="card">
                    <div class="card-body">
                        <h5 class="card-title">@FormTitle</h5>

                        <EditForm Model="form" OnValidSubmit="SubmitForm">
                            <DataAnnotationsValidator />
                            <ValidationSummary />

                            <div class="mb-2">
                                <label class="form-label">Title</label>
                                <InputText class="form-control" @bind-Value="form.Title" />
                            </div>

                            <div class="mb-2">
                                <label class="form-label">Description</label>
                                <InputTextArea class="form-control" @bind-
Value="form.Description" />
                            </div>

                            <div class="mb-2">
```

```
                                <label class="form-label">Location</label>
                                <InputText class="form-control" @bind-Value="form.Location" />
                            </div>

                            <div class="mb-2">
                                <label class="form-label">Start Date</label>
                                <InputDate class="form-control" @bind-Value="form.StartDate"
/>
                                <div class="form-text">If your API validates “future date”,
choose a date in the future.</div>
                            </div>

                            <div class="mb-3">
                                <label class="form-label">Base Ticket Price</label>
                                <InputNumber class="form-control" @bind-
Value="form.BaseTicketPrice" />
                            </div>

                            <button class="btn btn-primary"
type="submit">@SubmitButtonText</button>
                            <button class="btn btn-secondary ms-2" type="button"
@onclick="ResetForm">Cancel</button>
                        </EditForm>
                    </div>
                </div>
            </div>
        </div>
    }
</AuthGuard>

@code {
    private List<EventDto>? events;

    private string? success;
    private string? error;

    private long? editingId = null; // null => Create, not null => Edit
    private EventCreateUpdateDto form = NewForm();

    private string FormTitle => editingId is null ? "Create Event" : $"Edit Event (ID:
{editingId})";
    private string SubmitButtonText => editingId is null ? "Create" : "Save";

    protected override async Task OnInitializedAsync() {
        await Reload();
    }

    private static EventCreateUpdateDto NewForm()
        => new() { StartDate = DateTime.Now.AddDays(1), BaseTicketPrice = 1 };

    private void SetMessage(string? ok = null, string? err = null) {
        success = ok;
        error = err;
    }

    private async Task Reload() {
        SetMessage(null, null);

        var resp = await Http.GetAsync("api/events");
        if (resp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }

        if (!resp.IsSuccessStatusCode) {
            SetMessage(null, $"Load failed: {resp.StatusCode}");
```

```
            return;
        }

        events = await resp.Content.ReadFromJsonAsync<List<EventDto>>();
    }

    private void StartEdit(long id) {
        SetMessage(null, null);

        var e = events?.FirstOrDefault(x => x.EventId == id);
        if (e == null)
            return;

        editingId = id;

        form = new EventCreateUpdateDto {
            Title = e.Title,
            Description = e.Description,
            Location = e.Location,
            StartDate = e.StartDate,
            BaseTicketPrice = e.BaseTicketPrice
        };
    }

    private void ResetForm() {
        editingId = null;
        form = NewForm();
        SetMessage(null, null);
    }

    private async Task SubmitForm() {
        SetMessage(null, null);

        HttpResponseMessage resp;

        if (editingId is null) {
            resp = await Http.PostAsJsonAsync("api/events", form);
        } else {
            resp = await Http.PutAsJsonAsync($"api/events/{editingId}", form);
        }

        if (resp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }

        if (!resp.IsSuccessStatusCode) {
            SetMessage(null, $"Save failed: {resp.StatusCode}");
            return;
        }

        SetMessage(editingId is null ? "Event created" : "Event updated", null);

        ResetForm();
        await Reload();
    }

    private async Task DeleteEvent(long id) {
        if (!await Confirm($"Delete event #{id}?"))
            return;

        SetMessage(null, null);

        var resp = await Http.DeleteAsync($"api/events/{id}");

        if (resp.StatusCode == HttpStatusCode.Unauthorized) {
```

```
            Nav.NavigateTo("/login");
            return;
        }

        if (!resp.IsSuccessStatusCode) {
            SetMessage(null, $"Delete failed: {resp.StatusCode}");
            return;
        }

        SetMessage("Event deleted", null);

        if (editingId == id)
            ResetForm();
        await Reload();
    }

    private async Task<bool> Confirm(string message)
        => await JS.InvokeAsync<bool>("confirm", message);
      }
```

Лістинг коду TicketBooking.Client/Pages/Bookings.razor:

```
@page "/bookings"
@using System.Net
@using System.Net.Http.Json
@using TicketBooking.Client.Models
@inject HttpClient Http
@inject NavigationManager Nav
@inject IJSRuntime JS

<AuthGuard>
    <h3>Bookings</h3>

    @if (!string.IsNullOrWhiteSpace(success)) {
        <div class="alert alert-success">@success</div>
    }
    @if (!string.IsNullOrWhiteSpace(error)) {
        <div class="alert alert-danger">@error</div>
    }

    @if (bookings is null || events is null) {
        <p><em>Loading...</em></p>
    } else {
        <div class="row">
            <div class="col-12 col-lg-7">
                <table class="table table-striped align-middle">
                    <thead>
                        <tr>
                            <th>Customer</th>
                            <th>Email</th>
                            <th>Event</th>
                            <th class="text-center">Qty</th>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var b in bookings) {
                            <tr class="@(editingId == b.BookingId ? "table-warning" : "")">
                                <td>@b.CustomerName</td>
                                <td>@b.CustomerEmail</td>
                                <td>@b.EventTitle</td>
                                <td class="text-center">@b.Quantity</td>
                                <td class="text-end">
                                    <button class="btn btn-sm btn-outline-secondary me-2"
```

```
                                        @onclick='() =>
Nav.NavigateTo($"/bookings/{b.BookingId}")'>
                                    Details
                                </button>
                                <button class="btn btn-sm btn-outline-primary me-2"
                                        @onclick="() => StartEdit(b.BookingId)">
                                    Edit
                                </button>
                                <button class="btn btn-sm btn-outline-danger"
                                        @onclick="() => DeleteBooking(b.BookingId)">
                                    Delete
                                </button>
                            </td>
                        </tr>
                    }
                </tbody>
            </table>
        </div>

        <div class="col-12 col-lg-5">
            <div class="card">
                <div class="card-body">
                    <h5 class="card-title">@FormTitle</h5>

                    <EditForm Model="form" OnValidSubmit="SubmitForm">
                        <DataAnnotationsValidator />
                        <ValidationSummary />

                        <div class="mb-2">
                            <label class="form-label">Customer Name</label>
                            <InputText class="form-control" @bind-
Value="form.CustomerName" />
                        </div>

                        <div class="mb-2">
                            <label class="form-label">Customer Email</label>
                            <InputText class="form-control" @bind-
Value="form.CustomerEmail" />
                        </div>

                        <div class="mb-2">
                            <label class="form-label">Event</label>
                            <InputSelect class="form-select" @bind-Value="form.EventId">
                                <option value="0">-- select event --</option>
                                @foreach (var e in events) {
                                    <option value="@e.EventId">@e.Title
(@e.Location)</option>
                                }
                            </InputSelect>
                        </div>

                        <div class="mb-3">
                            <label class="form-label">Quantity</label>
                            <InputNumber class="form-control" @bind-Value="form.Quantity"
/>
                        </div>

                        <button class="btn btn-primary"
type="submit">@SubmitButtonText</button>
                        <button class="btn btn-secondary ms-2" type="button"
@onclick="ResetForm">Cancel</button>
                    </EditForm>
                </div>
            </div>
        </div>
    </div>
```

```
    }
</AuthGuard>

@code {
    private List<BookingDto>? bookings;
    private List<EventDto>? events;

    private string? success;
    private string? error;

    private long? editingId = null;
    private BookingCreateUpdateDto form = NewForm();

    private string FormTitle => editingId is null ? "Create Booking" : $"Edit Booking (ID:
{editingId})";
    private string SubmitButtonText => editingId is null ? "Create" : "Save";

    protected override async Task OnInitializedAsync() {
        await Reload();
    }

    private static BookingCreateUpdateDto NewForm()
        => new() { Quantity = 1 };

    private void SetMessage(string? ok = null, string? err = null) {
        success = ok;
        error = err;
    }

    private async Task Reload() {
        SetMessage(null, null);

        var evResp = await Http.GetAsync("api/events");
        if (evResp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }
        if (!evResp.IsSuccessStatusCode) {
            SetMessage(null, $"Load events failed: {evResp.StatusCode}");
            return;
        }
        events = await evResp.Content.ReadFromJsonAsync<List<EventDto>>();

        var bResp = await Http.GetAsync("api/bookings");
        if (bResp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }
        if (!bResp.IsSuccessStatusCode) {
            SetMessage(null, $"Load bookings failed: {bResp.StatusCode}");
            return;
        }
        bookings = await bResp.Content.ReadFromJsonAsync<List<BookingDto>>();
    }

    private void StartEdit(long id) {
        SetMessage(null, null);

        var b = bookings?.FirstOrDefault(x => x.BookingId == id);
        if (b == null)
            return;

        editingId = id;
        form = new BookingCreateUpdateDto {
            CustomerName = b.CustomerName,
            CustomerEmail = b.CustomerEmail,
```

```
            Quantity = b.Quantity,
            EventId = b.EventId
        };
    }

    private void ResetForm() {
        editingId = null;
        form = NewForm();
        SetMessage(null, null);
    }

    private async Task SubmitForm() {
        SetMessage(null, null);

        HttpResponseMessage resp;

        if (editingId is null) {
            resp = await Http.PostAsJsonAsync("api/bookings", form);
        } else {
            resp = await Http.PutAsJsonAsync($"api/bookings/{editingId}", form);
        }

        if (resp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }

        if (!resp.IsSuccessStatusCode) {
            SetMessage(null, $"Save failed: {resp.StatusCode}");
            return;
        }

        SetMessage(editingId is null ? "Booking created" : "Booking updated", null);

        ResetForm();
        await Reload();
    }

    private async Task DeleteBooking(long id) {
        if (!await Confirm($"Delete booking #{id}?"))
            return;

        SetMessage(null, null);

        var resp = await Http.DeleteAsync($"api/bookings/{id}");

        if (resp.StatusCode == HttpStatusCode.Unauthorized) {
            Nav.NavigateTo("/login");
            return;
        }

        if (!resp.IsSuccessStatusCode) {
            SetMessage(null, $"Delete failed: {resp.StatusCode}");
            return;
        }

        SetMessage("Booking deleted", null);

        if (editingId == id)
            ResetForm();
        await Reload();
    }

    private async Task<bool> Confirm(string message)
        => await JS.InvokeAsync<bool>("confirm", message);
}
```

Форми створення/редагування у Blazor реалізовано через EditForm у поєднанні з DataAnnotationsValidator та ValidationSummary. Це забезпечує валідацію введених даних на клієнті перед відправкою запиту в API. При помилках користувач одразу бачить повідомлення, а запит не виконується.

Рис. 1.2. Клієнтська валідація форми в Blazor (Events).

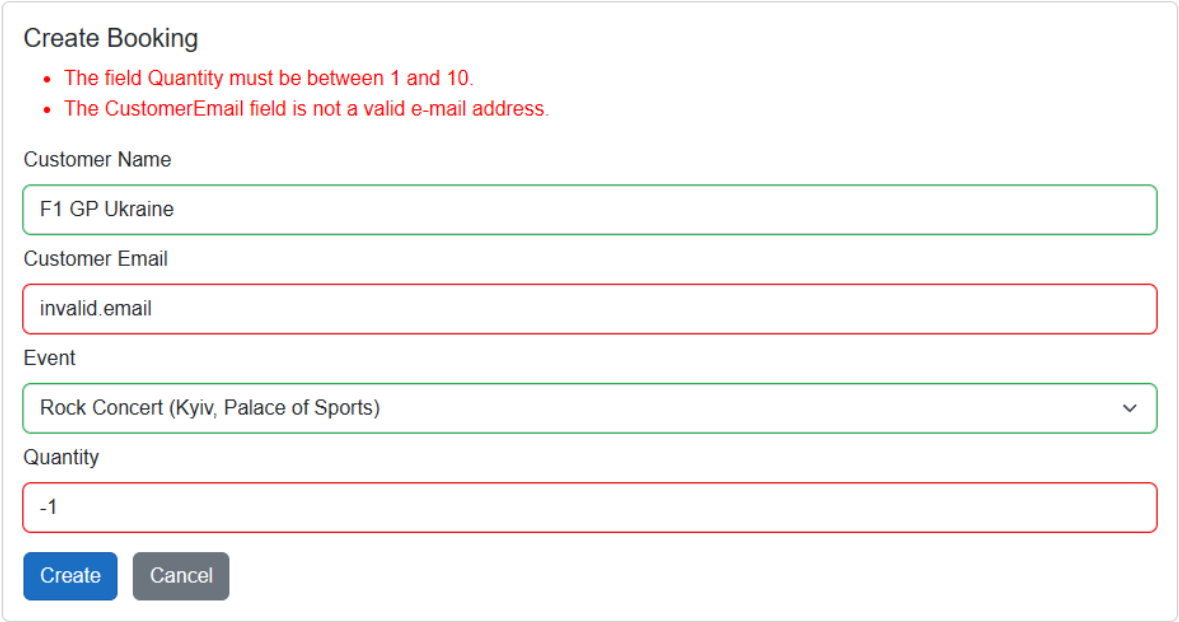

Рис. 1.3. Клієнтська валідація форми в Blazor (Bookings).

Для покращення UX було додано підтвердження видалення через виклик confirm(...) з використанням IJSRuntime, а також повідомлення про успіх або помилку у вигляді Bootstrap alert. Це зробило взаємодію зі сторінками більш зрозумілою та наближеною до реальних SPA-застосунків.

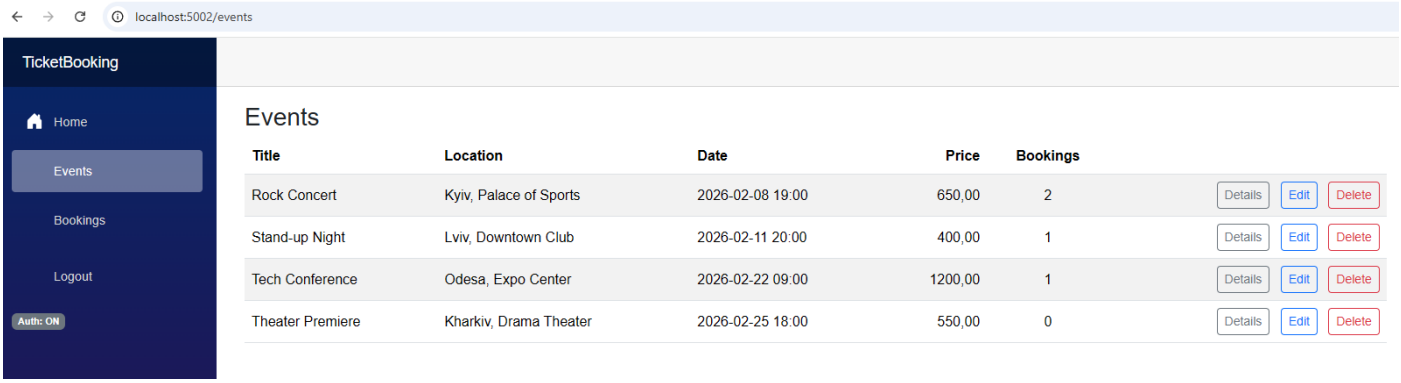

Рис. 1.4. CRUD-інтерфейс для Events у Blazor WebAssembly.

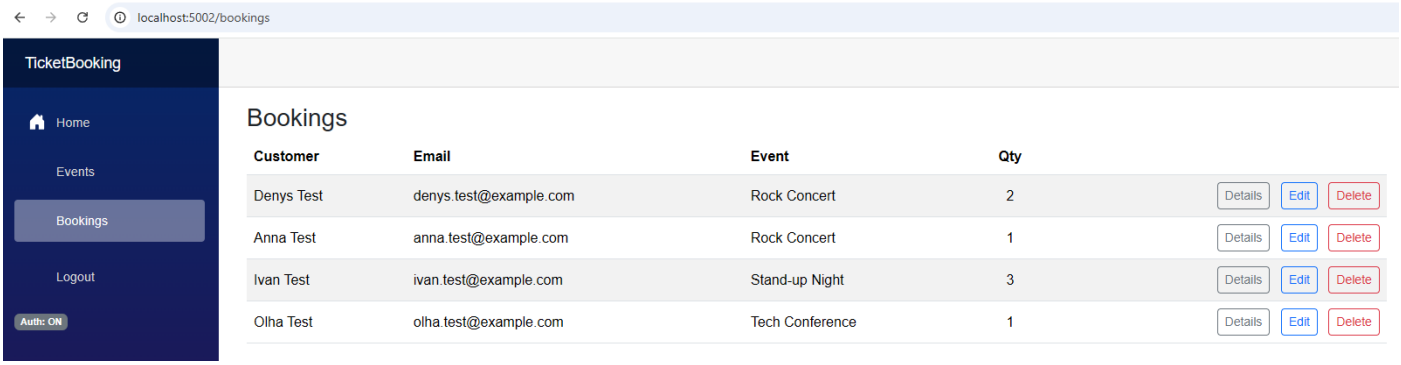

Рис. 1.5. CRUD-інтерфейс для Bookings у Blazor WebAssembly.

Окремо для реалізації сценарію «Read (Details)» створено сторінки деталей: /events/{id} та /bookings/{id}. На цих сторінках виконується запит до WebAPI за ідентифікатором, а дані відображаються у вигляді картки з повною інформацією про обраний запис.

Лістинг коду TicketBooking.Client/Pages/EventDetails.razor:

```
@page "/events/{id:long}"
@using System.Net
@using System.Net.Http.Json
@using TicketBooking.Client.Models
@inject HttpClient Http
@inject NavigationManager Nav
```

```
<AuthGuard>
    <h3>Event Details</h3>

    @if (!string.IsNullOrWhiteSpace(error))
    {
        <div class="alert alert-danger">@error</div>
    }

    @if (model is null && error is null)
    {
        <p><em>Loading...</em></p>
    }
    else if (model is not null)
    {
        <div class="card">
            <div class="card-body">
                <h5 class="card-title">@model.Title</h5>
                <p class="card-text">@model.Description</p>

                <dl class="row mb-0">
                    <dt class="col-sm-3">Location</dt>
                    <dd class="col-sm-9">@model.Location</dd>

                    <dt class="col-sm-3">Start</dt>
                    <dd class="col-sm-9">@model.StartDate.ToString("yyyy-MM-dd HH:mm")</dd>

                    <dt class="col-sm-3">Price</dt>
                    <dd class="col-sm-9">@model.BaseTicketPrice</dd>

                    <dt class="col-sm-3">Bookings</dt>
                    <dd class="col-sm-9">@model.BookingsCount</dd>
                </dl>
            </div>
        </div>

        <div class="mt-3">
            <button class="btn btn-secondary" @onclick='() =>
Nav.NavigateTo("/events")'>Back</button>
        </div>
    }
</AuthGuard>

    @code {
    [Parameter] public long id { get; set; }

    private EventDto? model;
    private string? error;

    protected override async Task OnParametersSetAsync()
    {
        error = null;
        model = null;

        var resp = await Http.GetAsync($"api/events/{id}");
        if (resp.StatusCode == HttpStatusCode.Unauthorized)
        {
            Nav.NavigateTo("/login");
            return;
        }

        if (resp.StatusCode == HttpStatusCode.NotFound)
        {
            Nav.NavigateTo("/not-found");
            return;
        }
```

```
        if (!resp.IsSuccessStatusCode)
        {
            error = $"Load failed: {resp.StatusCode}";
            return;
        }

        model = await resp.Content.ReadFromJsonAsync<EventDto>();
    }
}
```

Лістинг коду TicketBooking.Client/Pages/BookingDetails.razor:

```
@page "/bookings/{id:long}"
@using System.Net
@using System.Net.Http.Json
@using TicketBooking.Client.Models
@inject HttpClient Http
@inject NavigationManager Nav

<AuthGuard>
    <h3>Booking Details</h3>

    @if (!string.IsNullOrWhiteSpace(error))
    {
        <div class="alert alert-danger">@error</div>
    }

    @if (model is null && error is null)
    {
        <p><em>Loading...</em></p>
    }
    else if (model is not null)
    {
        <div class="card">
            <div class="card-body">
                <h5 class="card-title">Booking #@model.BookingId</h5>

                <dl class="row mb-0">
                    <dt class="col-sm-3">Customer</dt>
                    <dd class="col-sm-9">@model.CustomerName</dd>

                    <dt class="col-sm-3">Email</dt>
                    <dd class="col-sm-9">@model.CustomerEmail</dd>

                    <dt class="col-sm-3">Quantity</dt>
                    <dd class="col-sm-9">@model.Quantity</dd>

                    <dt class="col-sm-3">Event</dt>
                    <dd class="col-sm-9">@model.EventTitle</dd>

                    <dt class="col-sm-3">Created</dt>
                    <dd class="col-sm-9">@model.CreatedAtUtc.ToString("u")</dd>
                </dl>
            </div>
        </div>

        <div class="mt-3">
            <button class="btn btn-secondary" @onclick='() =>
Nav.NavigateTo("/bookings")'>Back</button>
        </div>
    }
</AuthGuard>
```

```
@code {
    [Parameter] public long id { get; set; }

    private BookingDto? model;
    private string? error;

    protected override async Task OnParametersSetAsync()
    {
        error = null;
        model = null;

        var resp = await Http.GetAsync($"api/bookings/{id}");
        if (resp.StatusCode == HttpStatusCode.Unauthorized)
        {
            Nav.NavigateTo("/login");
            return;
        }

        if (resp.StatusCode == HttpStatusCode.NotFound)
        {
            Nav.NavigateTo("/not-found");
            return;
        }

        if (!resp.IsSuccessStatusCode)
        {
            error = $"Load failed: {resp.StatusCode}";
            return;
        }

        model = await resp.Content.ReadFromJsonAsync<BookingDto>();
    }
}
```

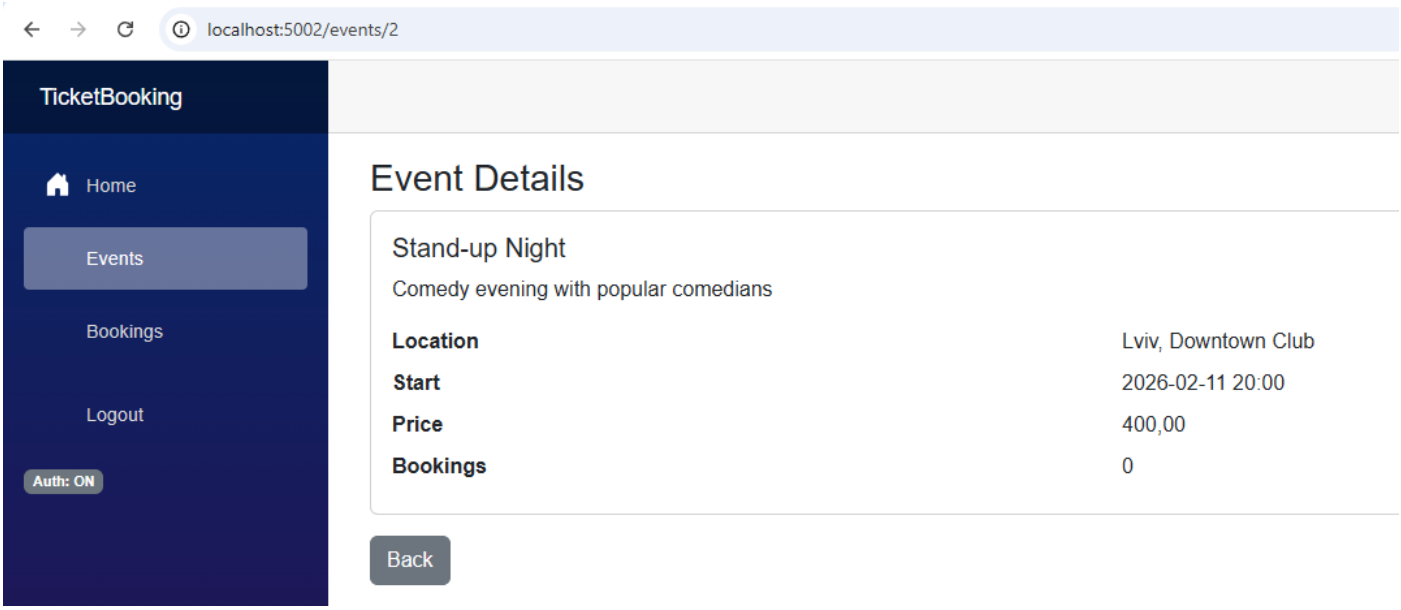

Рис. 1.6. Сторінки Details у Blazor для Event.

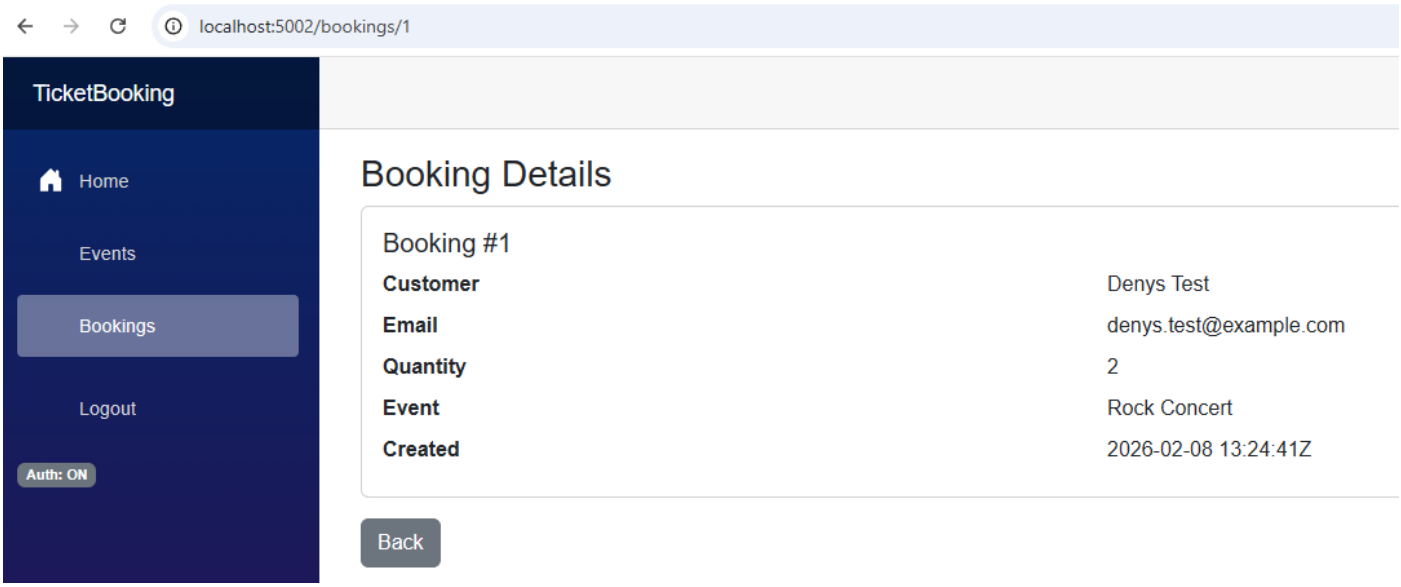

Рис. 1.7. Сторінки Details у Blazor для Booking.

Додатково у клієнті реалізовано сторінку Not Found (404) для невідомих маршрутів. Для цього було змінено App.razor (Router), де в секції NotFoundPage виводиться кастомний компонент NotFound.razor. Це дозволяє коректно обробляти помилкові URL у межах SPA.

Лістинг коду TicketBooking.Client/App.razor:

```
<Router AppAssembly="@typeof(App).Assembly" NotFoundPage="typeof(Pages.NotFound)">
    <Found Context="routeData">
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)"/>
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />
    </Found>
</Router>
```

Лістинг коду TicketBooking.Client/Pages/NotFound.razor:

```
@page "/not-found"
@inject NavigationManager Nav

<div class="text-center mt-5">
    <h1 class="display-6">404</h1>
    <p class="lead">Page not found.</p>

    <button class="btn btn-primary" @onclick='() => Nav.NavigateTo("/")'>Go Home</button>
</div>
```

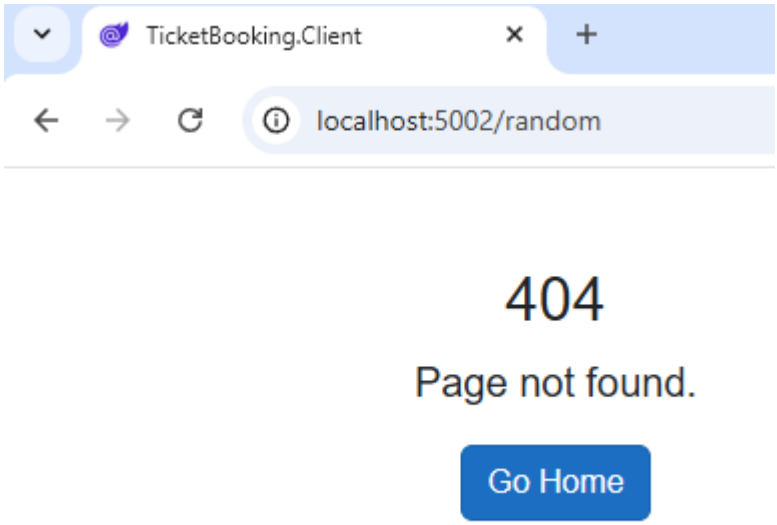

Рис. 1.8. Кастомна сторінка Not Found (404) у Blazor WebAssembly.

Завдання 2. Додайте автентифікацію та авторизацію до клієнта. Додайте сторінки та відповідні форми для реєстрації та входу. Реалізуйте автентифікацію на основі JWT. Приклад проекту, який реалізує автентифікацію та авторизацію за допомогою WebAPI та Blazor WebAssembly, можна знайти за наступним посиланням. Докладніше з механізмами автентифікації та авторизації в Blazor WebAssembly можна ознайомитись тут.

Оскільки ендпоїнти api/events та api/bookings у WebAPI захищені атрибутом [Authorize], клієнтський застосунок повинен виконувати автентифікацію й додавати JWT-токен до кожного запиту. Для цього у клієнті реалізовано процес входу: користувач вводить email та пароль на сторінці /login, після чого клієнт надсилає запит POST /api/auth/login. При успішній перевірці облікових даних сервер повертає токен, який зберігається у localStorage.

Лістинг коду TicketBooking.Client/Pages/Login.razor:

```
@page "/login"
@using System.ComponentModel.DataAnnotations
@using Microsoft.AspNetCore.WebUtilities
@inject TicketBooking.Client.Services.AuthService Auth
@inject NavigationManager Nav

<h3>Login</h3>

@if (!string.IsNullOrWhiteSpace(error)) {
    <div class="alert alert-danger">@error</div>
}
```

```
<EditForm Model="model" OnValidSubmit="DoLogin">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-2">
        <label>Email</label>
        <InputText class="form-control" @bind-Value="model.Email" />
    </div>

    <div class="mb-2">
        <label>Password</label>
        <InputText class="form-control" type="password" @bind-Value="model.Password" />
    </div>

    <button class="btn btn-primary" type="submit">Login</button>
</EditForm>

@code {
    private string? error;
    private string? returnUrl;

    private LoginForm model = new();

    private class LoginForm {
        [Required, EmailAddress]
        public string Email { get; set; } = "";

        [Required]
        public string Password { get; set; } = "";
    }

    protected override void OnInitialized() {
        var uri = Nav.ToAbsoluteUri(Nav.Uri);
        var query = QueryHelpers.ParseQuery(uri.Query);

        if (query.TryGetValue("returnUrl", out var ru))
            returnUrl = ru.ToString();
    }

    private async Task DoLogin() {
        error = null;

        var ok = await Auth.LoginAsync(model.Email, model.Password);
        if (!ok) {
            error = "Invalid email or password";
            return;
        }

        if (!string.IsNullOrWhiteSpace(returnUrl)) {
            Nav.NavigateTo(returnUrl);
        } else {
            Nav.NavigateTo("/events");
        }
    }
}
```

Лістинг коду: TicketBooking.Client/Services/AuthService.cs:

```
using Blazored.LocalStorage;
using System.Net.Http.Json;

namespace TicketBooking.Client.Services {
      public class AuthService {
```

```
        private readonly HttpClient _http;
        private readonly ILocalStorageService _localStorage;

        public AuthService(HttpClient http, ILocalStorageService localStorage) {
            _http = http;
            _localStorage = localStorage;
        }

        public class LoginRequest {
            public string Email { get; set; } = "";
            public string Password { get; set; } = "";
        }

        public class LoginResponse {
            public string Token { get; set; } = "";
            public DateTime Expires { get; set; }
        }
        public class RegisterRequest {
            public string Email { get; set; } = "";
            public string Password { get; set; } = "";
        }

        public async Task<bool> LoginAsync(string email, string password) {
            var resp = await _http.PostAsJsonAsync("api/auth/login", new LoginRequest
{
                Email = email,
                Password = password
            });

            if (!resp.IsSuccessStatusCode)
                return false;

            var data = await resp.Content.ReadFromJsonAsync<LoginResponse>();
            if (data == null || string.IsNullOrWhiteSpace(data.Token))
                return false;

            await _localStorage.SetItemAsync("authToken", data.Token);
            await _localStorage.SetItemAsync("authTokenExpires", data.Expires);

            return true;
        }

        public async Task LogoutAsync() {
            await _localStorage.RemoveItemAsync("authToken");
            await _localStorage.RemoveItemAsync("authTokenExpires");
        }

        public async Task<(bool Ok, string? Error)> RegisterAsync(string email, string
password) {
            var resp = await _http.PostAsJsonAsync("api/auth/register", new
RegisterRequest {
                Email = email,
                Password = password
            });

            if (resp.IsSuccessStatusCode)
                return (true, null);

            var msg = await resp.Content.ReadAsStringAsync();
            if (string.IsNullOrWhiteSpace(msg))
                msg = resp.StatusCode.ToString();

            return (false, msg);
        }

        public async Task<bool> IsAuthenticatedAsync() {
```

```
                    var token = await _localStorage.GetItemAsync<string>("authToken");
                    if (string.IsNullOrWhiteSpace(token))
                          return false;

                    var expires = await
_localStorage.GetItemAsync<DateTime?>("authTokenExpires");
                    if (expires is null)
                          return true;

                    return expires.Value.ToUniversalTime() > DateTime.UtcNow.AddSeconds(30);
              }
      }
}
```

Щоб токен автоматично додавався до всіх HTTP-запитів, було реалізовано DelegatingHandler (AuthHeaderHandler), який читає токен із localStorage та додає заголовок Authorization: Bearer <token> у кожен запит до API. Далі у Program.cs клієнта налаштовано HttpClientFactory так, щоб HttpClient використовував цей handler.

Лістинг коду TicketBooking.Client/Infrastructure/AuthHeaderHandler.cs:

```
using Blazored.LocalStorage;
using System.Net.Http.Headers;

namespace TicketBooking.Client.Infrastructure {
      public class AuthHeaderHandler : DelegatingHandler {

            private readonly ILocalStorageService _localStorage;

            public AuthHeaderHandler(ILocalStorageService localStorage) {
                  _localStorage = localStorage;
            }

            protected override async Task<HttpResponseMessage> SendAsync(
                  HttpRequestMessage request, CancellationToken cancellationToken) {
                  var token = await _localStorage.GetItemAsync<string>("authToken");
                  if (!string.IsNullOrWhiteSpace(token)) {
                        request.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", token);
                  }

                  return await base.SendAsync(request, cancellationToken);
            }
      }
}
```

Лістинг фрагменту коду TicketBooking.Client/Program.cs:

```
builder.Services.AddHttpClient("Api", client => {
      client.BaseAddress = new Uri("http://localhost:5001/");
})
.AddHttpMessageHandler<AuthHeaderHandler>();

builder.Services.AddScoped(sp =>
            sp.GetRequiredService<IHttpClientFactory>().CreateClient("Api"));
```

Для виходу користувача реалізовано сторінку /logout, яка очищає токен у localStorage та переадресовує користувача на сторінку входу. У навігаційному меню реалізовано динамічне відображення пунктів Login/Logout залежно від того, чи є токен у сховищі. Щоб меню оновлювалося при навігації без перезавантаження сторінки, компонент NavMenu.razor підписується на подію NavigationManager.LocationChanged і при зміні маршруту оновлює стан авторизації.

Лістинг коду: TicketBooking.Client/Pages/Logout.razor:

```
@page "/logout"
@inject TicketBooking.Client.Services.AuthService Auth
@inject NavigationManager Nav

<p>Signing out...</p>

@code {
    protected override async Task OnInitializedAsync() {
        await Auth.LogoutAsync();
        Nav.NavigateTo("/login?returnUrl=/", true);
    }
}
```

Лістинг коду TicketBooking.Client/Layout/NavMenu.razor:

```
@using TicketBooking.Client.Services
@implements IDisposable
@inject AuthService Auth
@inject NavigationManager Nav

<div class="top-row ps-3 navbar navbar-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="">TicketBooking</a>
        <button title="Navigation menu" class="navbar-toggler" @onclick="ToggleNavMenu">
            <span class="navbar-toggler-icon"></span>
        </button>
    </div>
</div>

<div class="@NavMenuCssClass nav-scrollable" @onclick="ToggleNavMenu">
    <nav class="nav flex-column">

        <div class="nav-item px-3">
            <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
                <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span>
                Home
            </NavLink>
        </div>

        <div class="nav-item px-3">
            <NavLink class="nav-link" href="events">
                <span class="bi bi-calendar-event-nav-menu" aria-hidden="true"></span>
                Events
            </NavLink>
        </div>

        <div class="nav-item px-3">
```

```
            <NavLink class="nav-link" href="bookings">
                <span class="bi bi-ticket-perforated-nav-menu" aria-hidden="true"></span>
                Bookings
            </NavLink>
        </div>

        <div class="nav-item px-3 mt-2">
            @if (isAuthenticated) {
                <NavLink class="nav-link" href="logout">
                    <span class="bi bi-box-arrow-right-nav-menu" aria-hidden="true"></span>
                    Logout
                </NavLink>
            } else {
                <NavLink class="nav-link" href="login">
                    <span class="bi bi-box-arrow-in-right-nav-menu" aria-hidden="true"></span>
                    Login
                </NavLink>
                <NavLink class="nav-link" href="register">
                    <span class="bi bi-person-plus-nav-menu" aria-hidden="true"></span>
                    Register
                </NavLink>
            }
        </div>

        <div class="nav-item px-3 mt-2">
            <span class="badge bg-secondary">Auth: @(isAuthenticated ? "ON" : "OFF")</span>
        </div>

    </nav>
</div>

@code {
    private bool collapseNavMenu = true;
    private bool isAuthenticated;

    private string? NavMenuCssClass => collapseNavMenu ? "collapse" : null;

    protected override async Task OnInitializedAsync() {
        Nav.LocationChanged += OnLocationChanged;
        await RefreshAuthState();
    }

    private async void OnLocationChanged(object? sender,
Microsoft.AspNetCore.Components.Routing.LocationChangedEventArgs e) {
        await RefreshAuthState();
        await InvokeAsync(StateHasChanged);
    }

    private async Task RefreshAuthState() {
        isAuthenticated = await Auth.IsAuthenticatedAsync();
    }

    private void ToggleNavMenu() {
        collapseNavMenu = !collapseNavMenu;
    }

    public void Dispose() {
        Nav.LocationChanged -= OnLocationChanged;
    }
}
```

Для запобігання доступу до сторінок CRUD без токена додано route-guard компонент AuthGuard. Він перевіряє наявність валідного токена перед ініціалізацією сторінки

та, якщо користувач не автентифікований, виконує редірект на /login. Для зручності користувача реалізовано повернення на початково запитану сторінку після входу за допомогою параметра returnUrl (наприклад, login?returnUrl=...).

Лістинг коду: TicketBooking.Client/Layout/AuthGuard.razor:

```
@inject TicketBooking.Client.Services.AuthService Auth
@inject NavigationManager Nav

@if (!checkedAuth) {
    <p><em>Checking authorization...</em></p>
} else {
    @ChildContent
}

@code {
    [Parameter] public RenderFragment? ChildContent { get; set; }

    private bool checkedAuth = false;

    protected override async Task OnInitializedAsync() {
        var ok = await Auth.IsAuthenticatedAsync();
        if (!ok) {
            var returnUrl = Uri.EscapeDataString(Nav.Uri);
            Nav.NavigateTo($"/login?returnUrl={returnUrl}");
            return;
        }

        checkedAuth = true;
    }
}
```

Лістинг коду Login.razor:

```
@page "/login"
@using System.ComponentModel.DataAnnotations
@using Microsoft.AspNetCore.WebUtilities
@inject TicketBooking.Client.Services.AuthService Auth
@inject NavigationManager Nav

<h3>Login</h3>

@if (!string.IsNullOrWhiteSpace(error)) {
    <div class="alert alert-danger">@error</div>
}

<EditForm Model="model" OnValidSubmit="DoLogin">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-2">
        <label>Email</label>
        <InputText class="form-control" @bind-Value="model.Email" />
    </div>

    <div class="mb-2">
        <label>Password</label>
        <InputText class="form-control" type="password" @bind-Value="model.Password" />
    </div>
```

```
    <button class="btn btn-primary" type="submit">Login</button>
</EditForm>

@code {
    private string? error;
    private string? returnUrl;

    private LoginForm model = new();

    private class LoginForm {
        [Required, EmailAddress]
        public string Email { get; set; } = "";

        [Required]
        public string Password { get; set; } = "";
    }

    protected override void OnInitialized() {
        var uri = Nav.ToAbsoluteUri(Nav.Uri);
        var query = QueryHelpers.ParseQuery(uri.Query);

        if (query.TryGetValue("returnUrl", out var ru))
            returnUrl = ru.ToString();
    }

    private async Task DoLogin() {
        error = null;

        var ok = await Auth.LoginAsync(model.Email, model.Password);
        if (!ok) {
            error = "Invalid email or password";
            return;
        }

        if (!string.IsNullOrWhiteSpace(returnUrl)) {
            Nav.NavigateTo(returnUrl);
        } else {
            Nav.NavigateTo("/events");
        }
    }
}
```

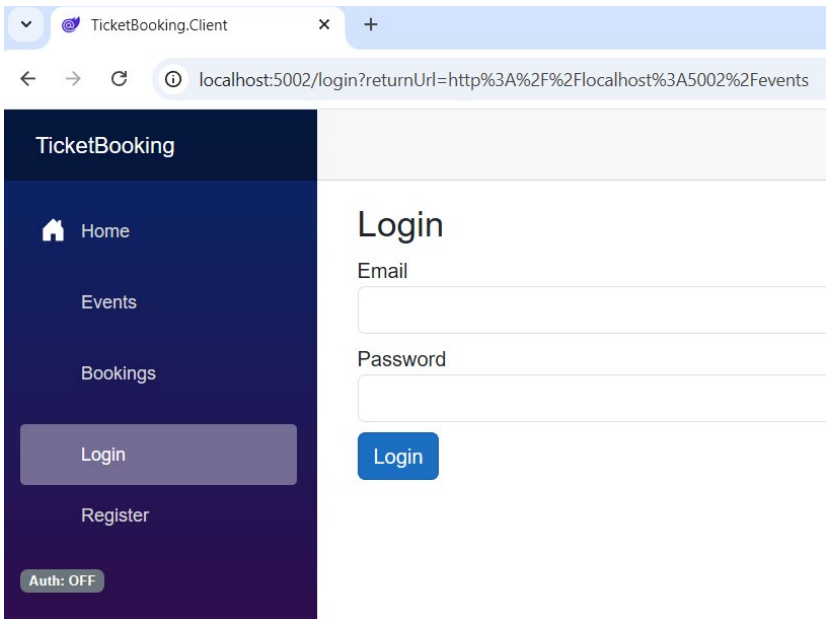

Рис. 2.1. Авторизаційна логіка клієнта: захист сторінок і повернення після входу.

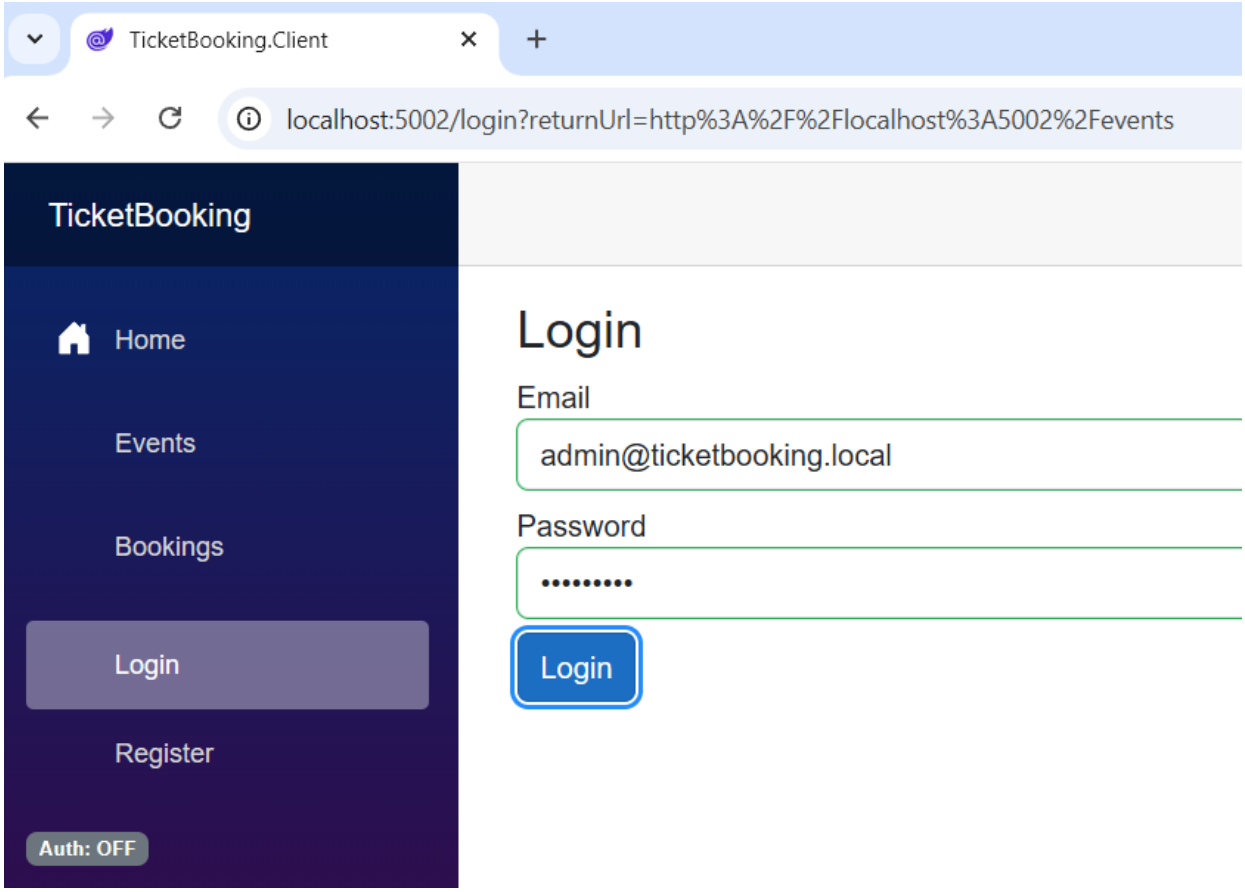

Рис. 2.2. Успішний вхід перед подальшим редіректом.

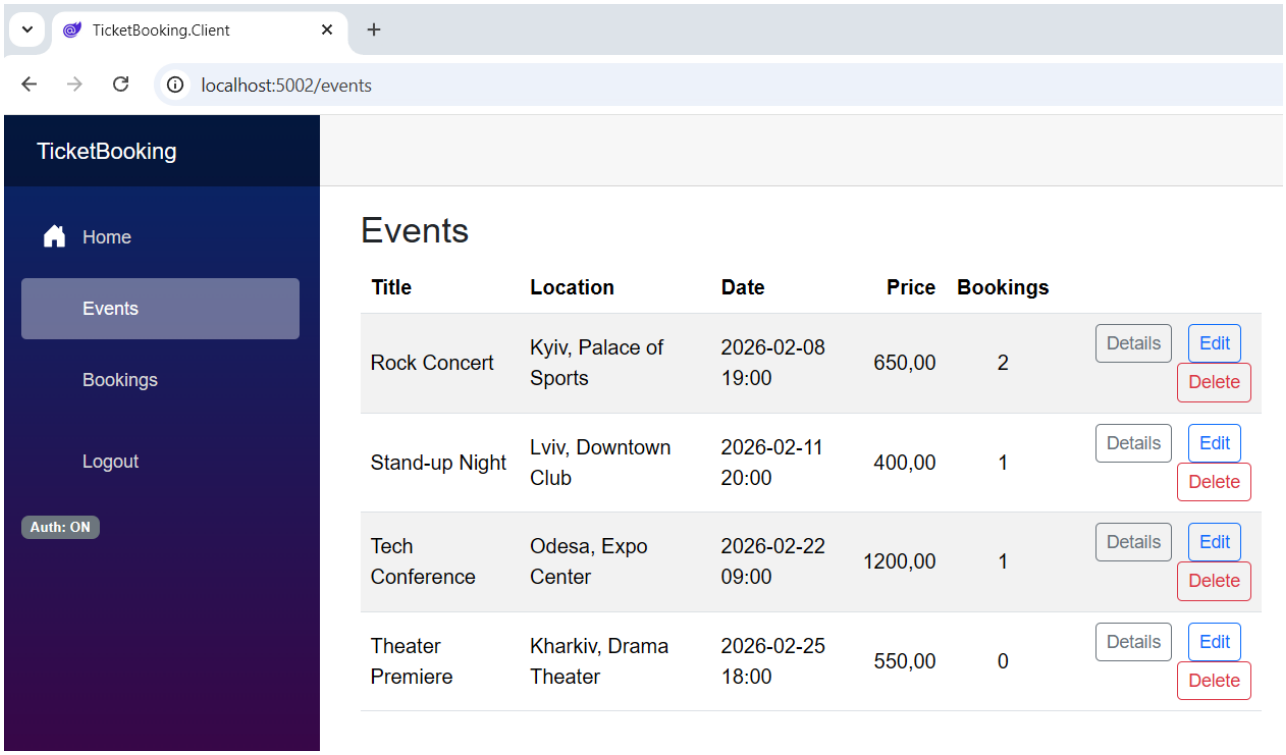

Рис. 2.3. Автоматичний редірект після успішного входу користувача.

Додатково реалізовано сторінку реєстрації /register, яка викликає endpoint POST /api/auth/register у WebAPI, виконує валідацію введених даних та дозволяє створити нового користувача без використання MVC-інтерфейсу.

Лістинг коду TicketBooking.Client/Pages/Register.razor:

```
@page "/register"
@using System.ComponentModel.DataAnnotations
@inject TicketBooking.Client.Services.AuthService Auth
@inject NavigationManager Nav

<h3>Register</h3>

@if (!string.IsNullOrWhiteSpace(success))
{
    <div class="alert alert-success">@success</div>
}

@if (!string.IsNullOrWhiteSpace(error))
{
    <div class="alert alert-danger">@error</div>
}

<EditForm Model="model" OnValidSubmit="DoRegister">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-2">
        <label>Email</label>
        <InputText class="form-control" @bind-Value="model.Email" />
    </div>

    <div class="mb-2">
        <label>Password</label>
        <InputText class="form-control" type="password" @bind-Value="model.Password" />
    </div>

    <div class="mb-3">
        <label>Confirm password</label>
        <InputText class="form-control" type="password" @bind-Value="model.ConfirmPassword" />
    </div>

    <button class="btn btn-primary" type="submit">Create account</button>
    <button class="btn btn-link" type="button" @onclick='() =>
Nav.NavigateTo("/login")'>Already have an account?</button>
</EditForm>

@code {
    private string? error;
    private string? success;

    private RegisterForm model = new();

    private class RegisterForm
    {
        [Required, EmailAddress]
        public string Email { get; set; } = "";

        [Required, MinLength(8, ErrorMessage = "Password must be at least 8 characters")]
        public string Password { get; set; } = "";
```

```
        [Required]
        [Compare(nameof(Password), ErrorMessage = "Passwords do not match")]
        public string ConfirmPassword { get; set; } = "";
    }

    private async Task DoRegister()
    {
        error = null;
        success = null;

        var (ok, err) = await Auth.RegisterAsync(model.Email, model.Password);
        if (!ok)
        {
            error = string.IsNullOrWhiteSpace(err) ? "Registration failed" : err;
            return;
        }

        success = "Registration successful. You can now log in.";
        Nav.NavigateTo("/login");
    }
}
```

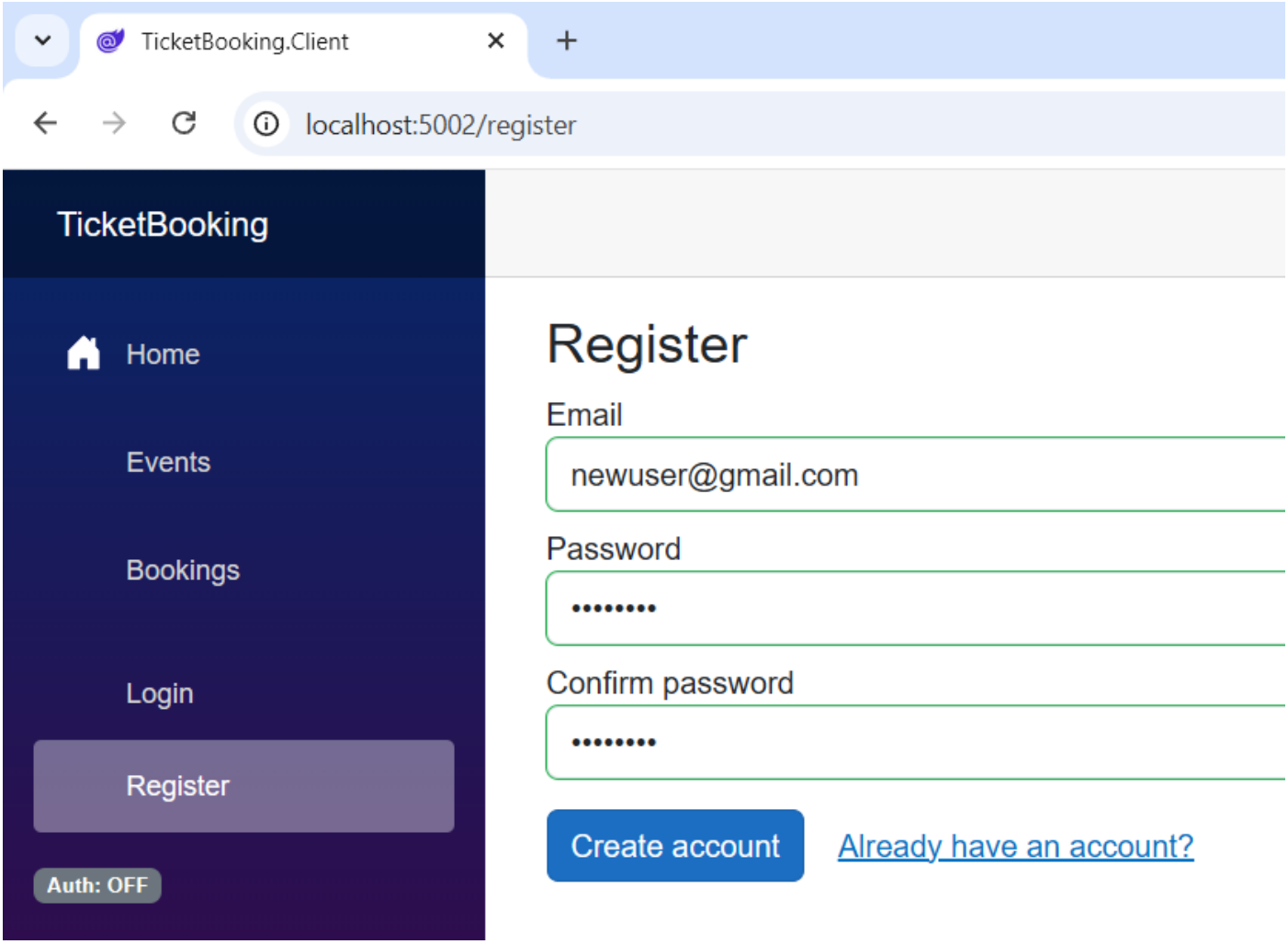

Рис. 2.4. Реєстрація користувача

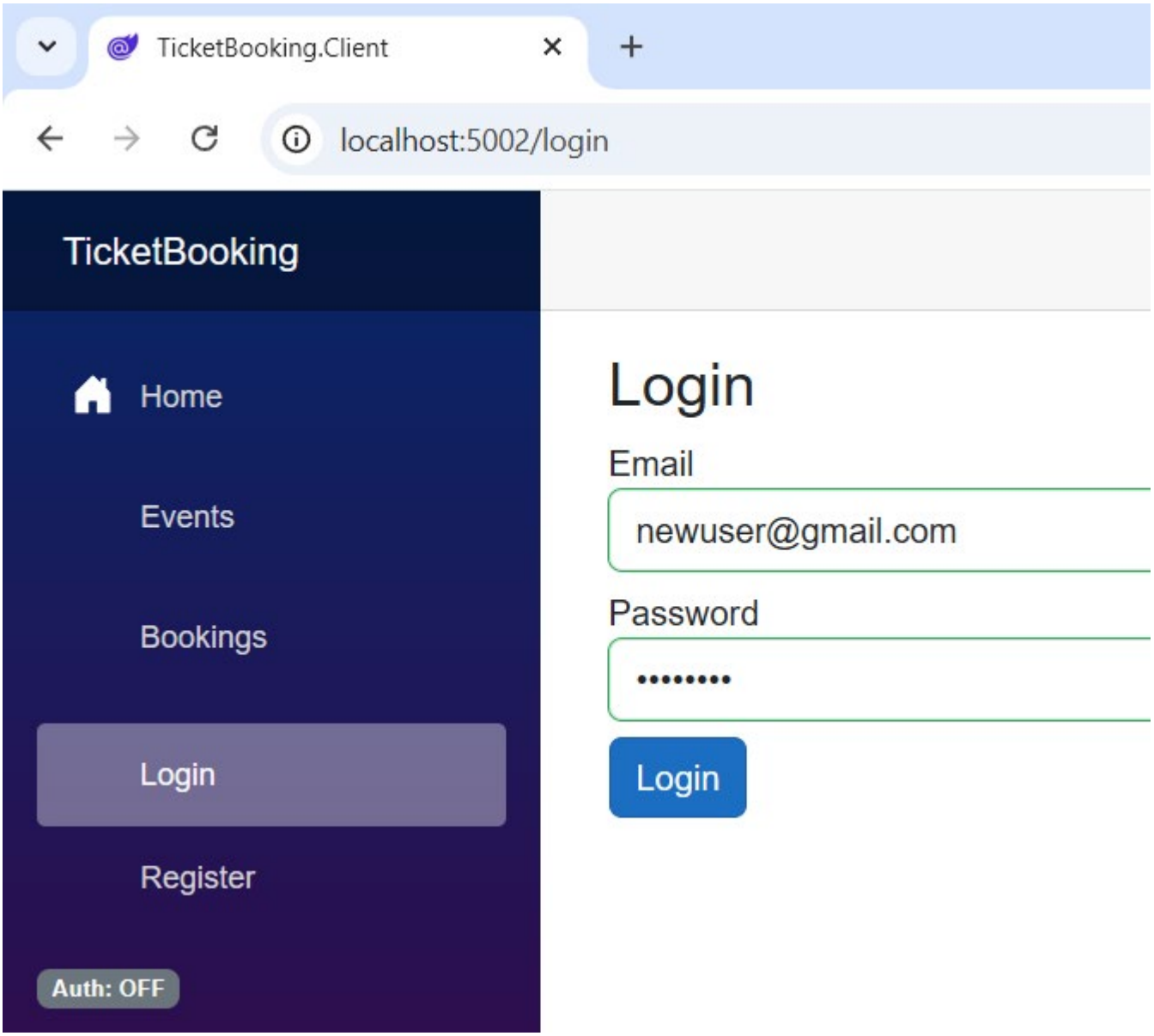

Рис. 2.5. Подальший редірект на сторінку логіну

Висновок: У ході лабораторної роботи №6 було створено клієнтський SPA-застосунок на Blazor WebAssembly, який інтегрується з RESTful WebAPI проєкту TicketBooking. На стороні WebAPI було увімкнено CORS, а в клієнті налаштовано HttpClient для виконання запитів до https://localhost:5001. Було реалізовано повний CRUD для сутностей Event і Booking з клієнтською валідацією форм та окремими сторінками Details. Для покращення UX додано підтвердження видалення, повідомлення про успіх/помилки, а також кастомну сторінку 404.

Оскільки WebAPI ендпоїнти захищені авторизацією, у клієнті реалізовано JWT-автентифікацію: сторінки Login/Register/Logout, збереження токена у localStorage, автоматичне додавання заголовка Authorization через DelegatingHandler, а також route-guard для захисту сторінок і механізм повернення користувача на початково запитану сторінку після успішного входу. Це забезпечило коректну і безпечну взаємодію Blazor-клієнта з WebAPI.